Java Level 11 — DSA & Problem Solving

Part 3 — Trees & Binary Search Trees

Status

Level 11: In Progress

Covered in this part:

- ✅ Binary Tree Fundamentals
- ✅ DFS
- ✅ Inorder Traversal
- ✅ Tree Recursion
- ✅ Maximum Depth
- ✅ Same Tree
- ✅ Invert Binary Tree
- ✅ Binary Search Tree Fundamentals
- ✅ Search in BST
- ✅ Validate BST
- ✅ Lowest Common Ancestor — BST
- ✅ BFS
- ✅ Level Order Traversal

---

1. Binary Tree Fundamentals

A Binary Tree is a tree where each node can have at most two children:

```
left
right
```

Example:

```
        10
       /
     5    15
    / \   /
   3   7 12  20
```

Java representation:

```java
static class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
    }
}
```

Unlike a Linked List:

```
1 → 2 → 3 → 4
```

a tree can branch:

```
        1
       /
      2   3
     /
    4   5
```

---

2. The Main Tree Question

For most Tree problems, first ask:

> How should I traverse the tree?

The two major approaches are:

```
DFS
→ Depth First Search

BFS
→ Breadth First Search
```

Mental model:

```
DFS
→ Go DEEP

BFS
→ Go LEVEL BY LEVEL
```

---

3. DFS — Depth First Search

DFS explores one branch deeply before moving to another branch.

The three classic DFS traversals are:

```
Preorder
Inorder
Postorder
```

The only major difference is:

> When do we process the ROOT?

---

4. Preorder Traversal

Order:

```
ROOT
LEFT
RIGHT
```

Example:

```
      1
     /
    2   3
   /
  4   5
```

Result:

```
1, 2, 4, 5, 3
```

Implementation:

```java
public static void preorder(TreeNode root) {
    if (root == null) {
        return;
    }

    System.out.println(root.val);

    preorder(root.left);
    preorder(root.right);
}
```

Mental model:

```
PROCESS
LEFT
RIGHT
```

---

5. Inorder Traversal

Order:

```
LEFT
ROOT
RIGHT
```

For:

```
        10
       /
      5    15
     /
    3   7
```

result:

```
3, 5, 7, 10, 15
```

Implementation from practice:

```java
public static List<Integer> inorderTraversal(
        TreeNode root
) {
    List<Integer> visited = new ArrayList<>();

    traverseInOrder(root, visited);

    return visited;
}

private static void traverseInOrder(
        TreeNode node,
        List<Integer> visited
) {
    if (node == null) {
        return;
    }

    traverseInOrder(node.left, visited);

    visited.add(node.val);

    traverseInOrder(node.right, visited);
}
```

Complexity:

```
Time  → O(n)
Space → O(h)
```

where:

```
n = number of nodes
h = height of tree
```

Balanced tree:

```
h ≈ log n
```

Worst-case skewed tree:

```
h = n
```

---

6. Important BST Property of Inorder

For a valid Binary Search Tree:

> Inorder traversal produces values in sorted order.

Example:

```
       8
      /
     3   10
    / \
   1   6    14
```

Inorder:

```
1, 3, 6, 8, 10, 14
```

This becomes useful when reasoning about BST validation and ordering.

---

7. Postorder Traversal

Order:

```
LEFT
RIGHT
ROOT
```

Implementation:

```java
public static void postorder(TreeNode root) {
    if (root == null) {
        return;
    }

    postorder(root.left);
    postorder(root.right);

    System.out.println(root.val);
}
```

Mental model:

```
LEFT
RIGHT
PROCESS
```

---

8. DFS Traversal Cheat Sheet

The easiest way to remember the three traversals is to move:

```
process(root);
```

Preorder

```
ROOT
LEFT
RIGHT
```

Inorder

```
LEFT
ROOT
RIGHT
```

Postorder

```
LEFT
RIGHT
ROOT
```

Remember:

```
PRE
→ root BEFORE children

IN
→ root IN BETWEEN children

POST
→ root AFTER children
```

---

9. Tree Recursion

Tree recursion follows an extremely important pattern:

```
Solve LEFT subtree

Solve RIGHT subtree

Combine their answers

Return answer to parent
```

Typical recursive structure:

```
if (root == null) {
    return baseValue;
}

var left = solve(root.left);
var right = solve(root.right);

return combine(root, left, right);
```

This mental model is more important than memorizing individual Tree solutions.

---

10. Maximum Depth of Binary Tree

Given:

```
        3
       /
      9   20
         /
        15  7
```

Maximum depth:

```
3
```

Recursive relationship:

```
depth(node)

=

1 + max(
    depth(left),
    depth(right)
)
```

Base case:

```
node == null
→ depth = 0
```

Implementation:

```
public static int maxDepth(TreeNode root) {
    if (root == null) {
        return 0;
    }

    return 1 + Math.max(
```

```
            maxDepth(root.left),
            maxDepth(root.right)
    );
}
```

Example reasoning:

```
depth(15) = 1
depth(7)  = 1

depth(20)
= 1 + max(1,1)
= 2

depth(9)
= 1

depth(3)
= 1 + max(1,2)
= 3
```

Complexity:

```
Time  → O(n)
Space → O(h)
```

---

11. Important Tree Recursion Mental Model

Maximum Depth demonstrates a very common pattern:

```
          NODE
         /
      LEFT    RIGHT
       ↓        ↓
    answer   answer
        \    /
         COMBINE
            ↓
        return up
```

Think:

What information should each subtree return to its parent?

This question solves many recursive Tree problems.

---

12. Same Tree

Determine whether two trees have:

```
Same structure
AND
Same values
```

Example:

```
Tree P          Tree Q

    1               1
   / \             /
  2   3           2   3

→ true
```

But:

```
    1               1
   /
  2                   2

→ false
```

---

13. Same Tree Base Cases

There are three important checks.

Both null

```
p == null
q == null
```

```
→ true
```

Only one null

```
p == null
OR
q == null

→ false
```

Different values

```
p.val != q.val

→ false
```

Otherwise recursively compare:

```
LEFT
AND
RIGHT
```

---

14. Same Tree Implementation

```java
public static boolean isSameTree(
        TreeNode p,
        TreeNode q
) {
    if (p == null && q == null) {
        return true;
    }

    if (p == null || q == null) {
        return false;
    }

    if (p.val != q.val) {
        return false;
    }
```

```
    return isSameTree(p.left, q.left)
            && isSameTree(p.right, q.right);
}
```

Mental model:

```
Current values same
        AND
Left subtree same
        AND
Right subtree same
```

Complexity:

```
Time  → O(n)
Space → O(h)
```

---

15. Important Same Tree Pitfall

A mistake from practice was using:

```
isSameTree(p.left, q.left)
==
isSameTree(p.right, q.right)
```

This is incorrect.

For example:

```
leftSame  = false
rightSame = false

false == false
→ true
```

But both subtrees being wrong does NOT make the trees equal.

We need:

```
leftSame && rightSame
```

Also remember to compare:

```
p.val == q.val
```

Structure alone is not enough.

---

16. Invert Binary Tree

Given:

```
       4
      /
     2   7
    / \ /
   1  3 6  9
```

Invert:

```
       4
      /
     7   2
    / \ /
   9  6 3  1
```

At every node:

```
swap left ↔ right
```

Then recursively invert both children.

---

17. Invert Tree Implementation

```
public static TreeNode invertTree(TreeNode root) {
    if (root == null) {
```

```
        return null;
    }

    TreeNode temp = root.left;

    root.left = root.right;
    root.right = temp;

    invertTree(root.left);
    invertTree(root.right);

    return root;
}
```

Mental model:

```
Current node

SWAP
left ↔ right

↓

Invert left

↓

Invert right
```

Complexity:

```
Time  → O(n)
Space → O(h)
```

---

18. Binary Search Tree — BST

A Binary Search Tree adds ordering rules to a Binary Tree.

For every node:

```
LEFT subtree values
<
node value

RIGHT subtree values
>
node value
```

Example:

```
       8
      /
     3   10
    / \
   1   6    14
      / \   /
     4   7 13
```

For root `8`:

```
everything left  < 8
everything right > 8
```

---

19. BST vs Binary Tree

A Binary Tree only guarantees:

```
At most two children
```

A BST additionally guarantees ordering:

```
left < node < right
```

That ordering allows us to eliminate entire subtrees while searching.

This is conceptually similar to:

```
Binary Search
```

---

20. Search in BST

Suppose we search for:

```
13
```

in:

```
        8

         10

           14
          /
         13
```

Reasoning:

```
13 > 8
→ RIGHT

13 > 10
→ RIGHT

13 < 14
→ LEFT

13 == 13
→ FOUND
```

---

21. Recursive BST Search

```
public static TreeNode searchBST(
        TreeNode root,
        int target
```

```
) {
    if (root == null) {
        return null;
    }

    if (root.val == target) {
        return root;
    }

    if (target < root.val) {
        return searchBST(
                root.left,
                target
        );
    }

    return searchBST(
            root.right,
            target
    );
}
```

Mental model:

```
Found?
→ return

Smaller?
→ LEFT

Greater?
→ RIGHT
```

---

22. Important BST Search Mistake

This only moves one level:

```
if (target < root.val) {
    root = root.left;
} else {
    root = root.right;
}
```

```
return root;
```

Example:

```
        8
       /
      3
     /
    1
```

Searching `1`:

```
8
→ move left
→ return 3 ❌
```

Recursive search must continue:

```
return searchBST(root.left, target);
```

or:

```
return searchBST(root.right, target);
```

---

23. Iterative BST Search

BST search can also be written without recursion:

```
public static TreeNode searchBST(
        TreeNode root,
        int target
) {
    while (root != null) {

        if (root.val == target) {
            return root;
        }
```

```
        if (target < root.val) {
            root = root.left;
        } else {
            root = root.right;
        }
    }

    return null;
}
```

Complexity:

```
Time  → O(h)
Space → O(1)
```

Balanced BST:

```
O(log n)
```

Worst-case skewed BST:

```
O(n)
```

---

24. Validate Binary Search Tree

This is an important BST interview problem.

Consider:

```
      5
     /
    3   8
       /
      2
```

At node `8`:

```
2 < 8
```

looks locally correct.

But the tree is NOT a valid BST.

Why?

Because `2` belongs to the:

```
RIGHT subtree of 5
```

Therefore:

```
2 must be > 5
```

but:

```
2 < 5
```

So BST validation cannot check only parent-child relationships.

---

25. Validate BST — Range Pattern

Each node receives a valid range:

```
(min, max)
```

The node must satisfy:

```
min < node.val < max
```

For the root:

```
(-∞, +∞)
```

For the left child:

```
(min, node.val)
```

For the right child:

```
(node.val, max)
```

---

26. Validate BST Example

For:

```
        5
       /
      3   8
         /
        6
```

ranges become:

```
5
→ (-∞, +∞)

3
→ (-∞, 5)

8
→ (5, +∞)

6
→ (5, 8)
```

Every node must stay within the constraints created by **all of its ancestors**.

---

27. Validate BST Implementation

```
public static boolean isValidBST(TreeNode root) {
    return valid(
            root,
```

```
            Long.MIN_VALUE,
            Long.MAX_VALUE
    );
}

private static boolean valid(
        TreeNode node,
        long min,
        long max
) {
    if (node == null) {
        return true;
    }

    if (node.val <= min || node.val >= max) {
        return false;
    }

    return valid(
            node.left,
            min,
            node.val
    ) && valid(
            node.right,
            node.val,
            max
    );
}
```

Complexity:

```
Time  → O(n)
Space → O(h)
```

---

28. Why Use `long` Bounds?

Avoid:

```
valid(
    root,
    Integer.MIN_VALUE,
```

```
    Integer.MAX_VALUE
);
```

Suppose:

```
root.val = Integer.MIN_VALUE
```

Then:

```
node.val <= min
```

becomes true even though the value itself may be valid.

Using:

```
Long.MIN_VALUE
Long.MAX_VALUE
```

creates bounds outside the entire `int` range.

---

29. Validate BST Mental Model

Don't ask:

```
Is left child smaller?

Is right child larger?
```

Ask:

```
What RANGE is this node allowed to be in?
```

Then propagate the range:

```
LEFT
→ upper bound becomes current value
```

```
RIGHT
→ lower bound becomes current value
```

---

30. Lowest Common Ancestor — BST

Given:

```
        6
       /
      2   8
     / \ /
    0  4 7  9
      /
     3   5
```

For:

```
p = 2
q = 8
```

LCA:

```
6
```

because this is the first point where their paths split.

---

31. BST LCA Observation

At current root:

```
Both p and q < root
→ LCA must be LEFT
```

```
Both p and q > root
→ LCA must be RIGHT
```

Otherwise:

```
One is on each side

OR

root is one of p/q
```

Therefore:

```
current root = LCA
```

---

32. Lowest Common Ancestor Implementation

```
public static TreeNode lowestCommonAncestor(
        TreeNode root,
        TreeNode p,
        TreeNode q
) {
    if (root == null) {
        return null;
    }

    if (p.val < root.val
            && q.val < root.val) {

        return lowestCommonAncestor(
                root.left,
                p,
                q
        );
    }

    if (p.val > root.val
            && q.val > root.val) {

        return lowestCommonAncestor(
                root.right,
                p,
                q
        );
    }
```

```
    return root;
}
```

Example:

```
p = 3
q = 5
```

Trace:

```
root = 6

3 < 6
5 < 6

→ LEFT
```

Then:

```
root = 2

3 > 2
5 > 2

→ RIGHT
```

Then:

```
root = 4

3 < 4
5 > 4

→ SPLIT
```

Therefore:

```
LCA = 4
```

---

33. LCA Complexity

```
Time  → O(h)
Space → O(h)
```

with recursion.

Balanced BST:

```
O(log n)
```

Skewed BST:

```
O(n)
```

---

34. BFS — Breadth First Search

DFS:

```
go deep
```

BFS:

```
go level by level
```

Example:

```
        3
       /
      9   20
         /
        15  7
```

BFS order:

```
Level 0 → 3

Level 1 → 9, 20

Level 2 → 15, 7
```

BFS uses a:

```
QUEUE
```

because Queue provides:

```
FIFO
```

---

35. Level Order Traversal

Expected result:

```
[
    [3],
    [9, 20],
    [15, 7]
]
```

Initialize:

```
Queue<TreeNode> queue =
        new ArrayDeque<>();

queue.offer(root);
```

Then process the tree one level at a time.

---

36. The Most Important BFS Trick

At the beginning of each level:

```
int levelSize = queue.size();
```

Then process exactly:

```
levelSize
```

nodes.

Why?

Suppose:

```
queue = [9,20]
```

Then:

```
levelSize = 2
```

While processing them, we might add:

```
15,7
```

to the queue.

But:

```
15,7
```

belong to the NEXT level.

Capturing `queue.size()` beforehand prevents us from mixing levels.

---

37. Level Order Implementation

```
public static List<List<Integer>> levelOrder(
        TreeNode root
) {
```

```
    List<List<Integer>> result =
            new ArrayList<>();

    if (root == null) {
        return result;
    }

    Queue<TreeNode> queue =
            new ArrayDeque<>();

    queue.offer(root);

    while (!queue.isEmpty()) {

        int levelSize = queue.size();

        List<Integer> currentLevel =
                new ArrayList<>();

        for (int i = 0; i < levelSize; i++) {

            TreeNode node = queue.poll();

            currentLevel.add(node.val);

            if (node.left != null) {
                queue.offer(node.left);
            }

            if (node.right != null) {
                queue.offer(node.right);
            }
        }

        result.add(currentLevel);
    }

    return result;
}
```

Complexity:

```
Time  → O(n)
Space → O(w)
```

where:

```
w = maximum width of tree
```

---

38. Level Order Trace

Tree:

```
        3
       /
      9   20
         /
        15  7
```

Initially:

```
queue = [3]

levelSize = 1
```

Process:

```
3
```

Add children:

```
queue = [9,20]
```

Result:

```
[[3]]
```

Next:

```
levelSize = 2
```

Process:

```
9
20
```

Add:

```
15
7
```

Queue:

```
[15,7]
```

Result:

```
[
 [3],
 [9,20]
]
```

Finally:

```
[15,7]
```

Result:

```
[
 [3],
 [9,20],
 [15,7]
]
```

---

39. Important BFS Mistakes

Mistake 1 — One List Per Node

This:

```
result.add(List.of(node.val));
```

produces:

```
[[3], [9], [20], [15], [7]]
```

That's BFS order, but it is **not level grouping**.

Instead create:

```
List<Integer> currentLevel =
        new ArrayList<>();
```

for each level.

Mistake 2 — Adding Null to ArrayDeque

Don't blindly:

```
queue.offer(node.left);
queue.offer(node.right);
```

`ArrayDeque` does not allow `null`.

Use:

```
if (node.left != null) {
    queue.offer(node.left);
}

if (node.right != null) {
    queue.offer(node.right);
}
```

Mistake 3 — Not Capturing Level Size

If you keep processing until the queue is empty inside the same level, newly added children will get mixed with the current level.

Capture:

```
int levelSize = queue.size();
```

before processing the level.

---

40. DFS vs BFS

DFS

Usually:

```
Recursion
or
Stack
```

Good for:

```
Subtree calculations
Depth
Path problems
Tree comparison
Tree transformation
BST validation
```

Mental model:

```
Go deep
```

BFS

Usually:

```
Queue
```

Good for:

```
Level-order traversal
Minimum number of edges/steps
Nearest node
Level calculations
Breadth-based processing
```

Mental model:

```
Go level by level
```

---

41. Tree Pattern Recognition Cheat Sheet

Traversal

If the problem asks:

```
Visit every node
Process nodes
Collect values
```

think:

```
DFS or BFS
```

Recursive Subtree Problem

If the answer depends on:

```
left subtree answer
+
right subtree answer
```

think:

```
Tree recursion
```

Template:

```
if (root == null) {
    return base;
}

var left = solve(root.left);
var right = solve(root.right);

return combine(left, right);
```

Middle / Level Processing

If you see:

```
level
nearest
minimum number of steps
breadth
```

think:

```
BFS + Queue
```

BST Search

If:

```
target < node
→ LEFT

target > node
→ RIGHT
```

think:

```
Use BST ordering
```

Don't unnecessarily traverse both subtrees.

BST Validation

If asked:

```
Is this a valid BST?
```

think:

```
Range constraints

(min, max)
```

not just:

```
left < parent < right
```

BST LCA

Think:

```
both left
→ go left

both right
→ go right

split
→ current node
```

---

42. Common Mistakes From Our Practice

Inorder Traversal

If the public method is:

```
static
```

the directly called helper must also be static unless you're calling it through an instance.

Same Tree

Don't forget:

```
p.val != q.val
```

And combine subtree results using:

```
&&
```

not:

```
==
```

BST Search

Moving:

```
root = root.left;
```

once isn't recursive search.

Either use:

```
return searchBST(root.left, target);
```

or an iterative `while`.

Validate BST

Don't only compare with the parent.

Ancestor constraints matter.

Use:

```
(min, max)
```

Validate BST Bounds

Prefer:

```
Long.MIN_VALUE
Long.MAX_VALUE
```

instead of integer extremes when node values are `int`.

BFS

Remember the combination:

```
Queue
+
levelSize
+
currentLevel
```

---

43. Complexity Cheat Sheet

Pattern / Problem	Time	Space
DFS Traversal	`O(n)`	`O(h)`
Inorder	`O(n)`	`O(h)`
Maximum Depth	`O(n)`	`O(h)`
Same Tree	`O(n)`	`O(h)`
Invert Tree	`O(n)`	`O(h)`
BST Search	`O(h)`	`O(h)` recursive

Pattern / Problem	Time	Space
Iterative BST Search	`O(h)`	`O(1)`
Validate BST	`O(n)`	`O(h)`
BST LCA	`O(h)`	`O(h)` recursive
Level Order BFS	`O(n)`	`O(w)`

Where:

```
n = number of nodes
h = tree height
w = maximum tree width
```

---

44. Tree Complexity Mental Model

For DFS recursion:

```
Space ≈ tree height
```

Balanced:

```
      ●
     /
    ●    ●
   / \   /

height ≈ log n
```

So:

```
O(log n)
```

A skewed tree:

```
●
  ●
```

```
    •
      •
```

has:

```
height ≈ n
```

So recursion stack:

```
O(n)
```

---

45. Revision Checklist

Binary Tree Fundamentals

- [x] TreeNode structure
- [x] Binary Tree terminology
- [x] DFS
- [x] BFS

DFS Traversal

- [x] Preorder concept
- [x] Inorder implementation
- [x] Postorder concept

Tree Recursion

- [x] Base case
- [x] Recursive subtree calls
- [x] Combine subtree answers
- [x] Maximum Depth

Tree Comparison / Transformation

- [x] Same Tree
- [x] Invert Binary Tree

BST

- [x] BST ordering property

- [x] Search BST
- [x] Validate BST
- [x] Range validation
- [x] Lowest Common Ancestor

BFS

- [x] Queue usage
- [x] Level size
- [x] Level Order Traversal

---

46. Level 11 Progress

```
PART 1

✅ Big-O

✅ Arrays & Hashing

✅ Two Pointers

✅ Sliding Window

✅ Prefix Sum

✅ Stack

✅ Monotonic Stack

✅ Queue / Deque Basics

🟡 Monotonic Deque — revisit


PART 2

✅ Binary Search

✅ Binary Search Boundaries

✅ Rotated Binary Search

✅ Binary Search on Answer

✅ Linked List Fundamentals
```

✅ Reverse Linked List

✅ Slow / Fast Pointers

✅ Cycle Detection

✅ Dummy Node

✅ Merge Sorted Lists

🟡 Remove Nth From End — implementation pending

PART 3

✅ Binary Tree Fundamentals

✅ DFS

✅ Tree Traversal

✅ Tree Recursion

✅ Maximum Depth

✅ Same Tree

✅ Invert Binary Tree

✅ BST Fundamentals

✅ BST Search

✅ Validate BST

✅ Lowest Common Ancestor

✅ BFS / Level Order

NEXT

⬜ Heap / Priority Queue

⬜ Graphs

⬜ Graph BFS / DFS

⬜ Topological Sort

⬜ Union Find

⬜ Recursion

⬜ Backtracking

⬜ Greedy

⬜ Dynamic Programming

⬜ Intervals

Final Tree Mental Model

When you see a Tree problem, don't immediately start writing recursion.

First classify it.

```
TREE PROBLEM
     │
     ├── Need to go deep?
     │
     │      → DFS
     │
     ├── Need level-by-level?
     │
     │      → BFS + Queue
     │
     ├── Need answers from children?
     │
     │      → Recursion
     │        solve left
     │        solve right
     │        combine
     │
     └── Is it a BST?
            │
```

```
            └── Use ordering
                to eliminate branches
```

Then remember the core patterns:

```
Traversal
→ DFS / BFS

Subtree calculation
→ recursion

Depth
→ 1 + max(left, right)

Compare trees
→ current + left + right

Transform tree
→ modify current + recurse

BST search
→ choose one branch

BST validation
→ carry allowed range

BST LCA
→ follow values until paths split

Level order
→ Queue + levelSize
```

The most important shift with Trees is:

> Don't try to understand the entire tree at once.

At each node ask:

```
What should I do with THIS node?

What should the LEFT subtree return?

What should the RIGHT subtree return?
```

```
How do I combine those answers?
```

Once that becomes natural, a large number of Tree interview problems become variations of the same few patterns.